

Universidad de Antioquia

Facultad de Ingeniería

Ingeniería Electrónica

Invincible Kickboxer

Informe técnico del proyecto final

Asignatura: Informática II

Semestre: 2026-1

Estudiantes:

Mauricio Reyes Rosero

Andrés Guerrero

Medellín, junio de 2026

Índice

1. Introducción	2
2. Arquitectura general	2
2.1. Diagrama de jerarquía de clases	2
3. Capa de entidades	3
3.1. Clase <code>Entidad</code> (abstracta)	3
3.1.1. Atributos principales	3
3.1.2. Métodos principales	3
3.2. Clase <code>Jugador</code>	3
3.2.1. Atributos principales	4
3.2.2. Constantes de diseño	4
3.2.3. Métodos principales	4
3.3. Clase <code>Enemigo</code> (abstracta)	5
3.4. Clase <code>AngstromLevy</code>	5
3.4.1. Estados de la IA	5
3.4.2. Atributos cognitivos principales	5
3.4.3. Física del golpe	6
3.5. Clase <code>Variante</code>	6
3.6. Clase <code>VariantePortal</code>	7
3.7. Clase <code>Portal</code>	7
4. Capa de servicios	7
4.1. Clase <code>GameEngine</code>	8
4.2. Clase <code>PhysicsEngine</code>	8
4.3. Clase <code>DifficultyConfig</code>	9
4.4. Clase <code>GameState</code>	9
5. Capa de presentación	9
5.1. Clase <code>MainWindow</code>	9
6. Estructuras de datos	10
7. Conclusiones	10

1. Introducción

Invincible Kickboxer es un videojuego de acción en dos dimensiones desarrollado en C++ con el framework Qt, como proyecto final de la asignatura Informática II. El juego consiste en dos niveles: en el nivel 1 el jugador se enfrenta al jefe Angstrom Levy, un agente de inteligencia artificial con comportamiento adaptativo; en el nivel 2 debe sobrevivir a un enjambre de variantes clonadas que aparecen desde portales dinámicos.

El objetivo del informe es documentar la arquitectura del proyecto desde el punto de vista de la programación orientada a objetos, describiendo las clases implementadas, sus atributos, sus métodos más relevantes, las relaciones de herencia y las estructuras de datos utilizadas.

El proyecto está alojado en GitHub bajo el repositorio `Andresdgr/Proyecto-Final`, rama `main`.

2. Arquitectura general

El proyecto sigue una separación clara en tres capas:

1. **Capa de entidades:** jerarquía de clases que representan los objetos del juego (jugador, enemigos, portales). Todas heredan de la clase base `Entidad`.
2. **Capa de servicios:** clases que gestionan la lógica sin tener representación gráfica propia (`GameEngine`, `PhysicsEngine`, `GameState`, `DifficultyConfig`).
3. **Capa de presentación:** la clase `MainWindow`, única clase que interactúa directamente con Qt para mostrar sprites, HUD y menús.

Esta separación garantiza que la lógica del juego sea completamente independiente de la interfaz gráfica. `GameEngine` no tiene ningún puntero a `QGraphicsScene`; simplemente ejecuta la lógica y expone su estado a través de `GameState`.

2.1. Diagrama de jerarquía de clases

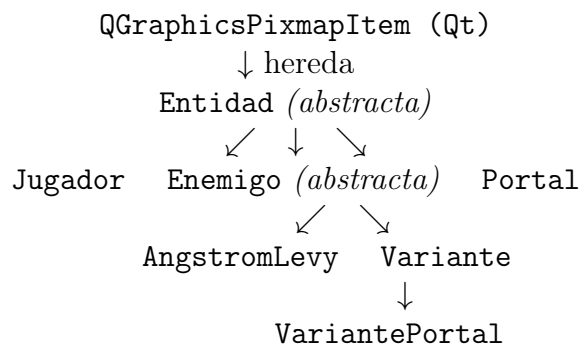


Figura 1: Jerarquía de herencia del proyecto

3. Capa de entidades

3.1. Clase Entidad (abstracta)

Entidad es la raíz de la jerarquía de objetos del juego. Hereda de `QGraphicsPixmapItem`, lo que permite que cada objeto del juego sea simultáneamente un elemento lógico con estado físico y un elemento visual renderizable en la escena de Qt.

3.1.1. Atributos principales

Atributo	Tipo	Descripción
<code>x, y</code>	<code>float</code>	Posición en el espacio 2D de la escena
<code>velX, velY</code>	<code>float</code>	Velocidad horizontal y vertical
<code>vida</code>	<code>float</code>	Puntos de vida actuales
<code>vidaMaxima</code>	<code>float</code>	Vida máxima al momento de creación
<code>masa</code>	<code>float</code>	Masa del objeto (afecta física de rebote)
<code>activo</code>	<code>bool</code>	Indica si el objeto sigue en juego
<code>tiempoRecuperacionGolpe</code>	<code>float</code>	Contador de invulnerabilidad tras recibir daño

Cuadro 1: Atributos protegidos de **Entidad**

3.1.2. Métodos principales

- `virtual void update(float dt) = 0` — Método virtual puro. Obliga a toda clase concreta a implementar su ciclo de vida por frame.
- `virtual void recibirDanio(float cantidad)` — Resta vida, activa el timer de recuperación y desactiva el objeto si la vida llega a cero.
- `void setPosicion(float x, float y)` — Actualiza la posición lógica y sincroniza `setPos()` de Qt para mover el sprite en pantalla.
- `void limitarBordes(...)` — Restringe la posición dentro de los límites de la pantalla.

3.2. Clase Jugador

Jugador es la clase concreta que representa al personaje controlado por el usuario. Hereda directamente de **Entidad**.

3.2.1. Atributos principales

Atributo	Tipo	Descripción
velBase	float	Velocidad base de movimiento (200 px/s)
cargando	bool	Indica si el jugador está cargando un golpe
tiempoCarga	float	Tiempo acumulado de carga
hitStreak	uint8_t	Contador de golpes consecutivos
comboActivo	bool	Indica si el combo de 3 golpes está listo
invulnerable	bool	Indica si el jugador está en periodo de gracia
tiempoInvulnerable	float	Tiempo restante de invulnerabilidad
danioActual	float	Daño calculado del último ataque

Cuadro 2: Atributos privados de Jugador

3.2.2. Constantes de diseño

Constante	Valor	Descripción
UMBRAL_CARGA	0.4 s	Tiempo mínimo para activar golpe cargado
FACTOR_DANO_CARGA	2.0	Multiplicador de daño con golpe cargado
DURACION_INVUL	0.8 s	Duración de la invulnerabilidad post-golpe
GOLPES_COMBO	3	Golpes consecutivos para activar combo

Cuadro 3: Constantes estáticas de Jugador

3.2.3. Métodos principales

- `void moverX(float dir) / moverY(float dir)` — Ajustan `velX` y `velY` según la dirección. No tienen efecto si el jugador está cargando.
- `void iniciarCarga()` — Activa el flag `cargando` y comienza a contar `tiempoCarga`. Mientras carga, `velX = 0`.
- `void atacar()` — Calcula el daño según si el tiempo de carga supera el umbral o si el combo está activo.
- `void recibirDanio(float cantidad)` — Sobreescribe el método base. Si el jugador es invulnerable, ignora el daño.
- `void recibirDanioAmbiental(float cantidad)` — Daño del portal que ignora la invulnerabilidad.

3.3. Clase `Enemigo` (abstracta)

`Enemigo` es una capa de abstracción intermedia entre `Entidad` y los enemigos concretos.

- `float danio` — Daño que inflige al jugador en cada colisión.
- `uint16_t puntosAlDerrotar` — Puntos que se suman al marcador cuando el enemigo es derrotado.
- `virtual void update(float dt, const Jugador& jugador) = 0` — Segundo método virtual puro.
- `virtual void moverse(float dt) = 0` — Método virtual puro para el desplazamiento físico.

3.4. Clase `AngstromLevy`

`AngstromLevy` es el jefe del nivel 1. Implementa un agente de inteligencia artificial de cuatro componentes.

3.4.1. Estados de la IA

```

1 enum class EstadoIA : uint8_t {
2     AGRESIVO    = 0,
3     EVASIVO     = 1,
4     DEFENSIVO   = 2,
5     GOLPEADO    = 3
6 };

```

Listing 1: Estados del agente `AngstromLevy`

3.4.2. Atributos cognitivos principales

Atributo	Tipo	Descripción
<code>estadoActual</code>	<code>EstadoIA</code>	Estado activo de la máquina de estados
<code>historialDirecciones</code>	<code>deque<int8_t></code>	Ventana deslizante de 10 direcciones
<code>agresividad</code>	<code>float</code>	Multiplicador de velocidad
<code>radioPortal</code>	<code>float</code>	Radio de teleportación
<code>contadorInteracciones</code>	<code>uint16_t</code>	Número de frames procesados
<code>velocidadBase</code>	<code>float</code>	Velocidad base configurable
<code>cooldownEvasionBase</code>	<code>float</code>	Tiempo mínimo entre evasiones

Cuadro 4: Atributos cognitivos de `AngstromLevy`

1. Percibir Captura posición, vida y dirección del jugador, y calcula $d = \sqrt{(\Delta x)^2 + (\Delta y)^2}$.

2. Razonar

```

1 void AngstromLevy::razonar() {
2     if (estadoActual == EstadoIA::GOLPEADO) return;
3     if (cooldownEvasion > 0.0f) {
4         estadoActual = EstadoIA::AGRESIVO; return;
5     }
6     if (vida < vidaMaxima * UMBRAL_VIDA_DEFENSIVO) {
7         estadoActual = EstadoIA::DEFENSIVO; return;
8     }
9     if (jugadorCercano) {
10        estadoActual = EstadoIA::EVASIVO; return;
11    }
12    estadoActual = EstadoIA::AGRESIVO;
13 }

```

Listing 2: Lógica de razonamiento de Levy

3. Actuar AGRESIVO $\rightarrow$ moverse(dt), EVASIVO $\rightarrow$ teleportarse(), DEFENSIVO $\rightarrow$ esperarYDanar().

4. Aprender

```

1 void AngstromLevy::aprender() {
2     historialDirecciones.push_front(direccionPercibida);
3     if (historialDirecciones.size() > CAPACIDAD_HISTORIAL)
4         historialDirecciones.pop_back();
5     if (contadorInteracciones % INTERVALO_APRENDIZAJE == 0)
6         ajustarParametros();
7 }

```

Listing 3: Ventana deslizante de aprendizaje

La agresividad crece con el tiempo:

$$\text{agresividad} = 1,0 + \frac{\text{contadorInteracciones}}{300}, \quad \text{máximo } 2,0 \quad (1)$$

3.4.3. Física del golpe

$$x(t) = x_0 + v_{0x} \cdot t \quad (2)$$

$$y(t) = y_0 + v_{0y} \cdot t + \frac{1}{2}g \cdot t^2 \quad (3)$$

3.5. Clase Variante

```

1 enum class Estado { RODEAR, ATACAR };

```

Listing 4: Estados de Variante

Atributo	Tipo	Descripción
estado	Estado	Estado actual (RODEAR / ATACAR)
angulo	float	Ángulo en la órbita alrededor del jugador
tiempoEnEstado	float	Tiempo acumulado en el estado actual
velRodear	float	Velocidad al orbitar
velAtacar	float	Velocidad al cargar

Cuadro 5: Atributos de Variante

3.6. Clase VariantePortal

Atributo	Tipo	Descripción
pesosZonas[9]	float []	Peso aprendido por zona de la cuadrícula 3x3
zonaActualJugador	int	Zona donde está el jugador
alpha	float	Tasa de aprendizaje (0.3)

Cuadro 6: Miembros estáticos de VariantePortal

$$\text{peso}[z] = \text{peso}[z] \cdot (1 - \alpha) + \text{recompensa} \cdot \alpha, \quad \alpha = 0,3 \quad (4)$$

3.7. Clase Portal

Constructor nivel 1 — MAS:

$$\text{radio}(t) = A \cdot \sin(\omega \cdot t) \quad (5)$$

Constructor nivel 2 — Orbital:

$$x(t) = c_x + r \cdot \cos(\omega t + \varphi) \quad (6)$$

$$y(t) = c_y + r \cdot \sin(\omega t + \varphi) \cdot 0,7 \quad (7)$$

4. Capa de servicios

4.1. Clase GameEngine

Atributo	Tipo	Descripción
jugador	Jugador*	Puntero al jugador activo
enemigos	QList<Enemigo*>	Lista polimórfica de enemigos
portales	QList<Portal*>	Lista de portales activos
physics	PhysicsEngine	Motor de física
estado	GameState	Estado observable de la partida
config	DifficultyConfig	Configuración de dificultad
nivelActivo	uint8_t	Nivel en ejecución (1 o 2)

Cuadro 7: Atributos privados de GameEngine

```

1 for (Enemigo* e : std::as_const(enemigos)) {
2     if (e->isActivo() && jugador) {
3         e->update(dt, *jugador);
4     }
5 }

```

Listing 5: Despacho polimórfico en nivel 1

4.2. Clase PhysicsEngine

ID	Nombre	Uso en el juego
F1	Movimiento parabólico	Trayectoria de Levy al recibir un golpe
F2	Movimiento armónico simple	Radio oscilante del portal en nivel 1
F3	Aceleración progresiva	Velocidad creciente de los clones
F4	Rebote contra borde	Reflexión de velocidad al tocar el borde
F5	Movimiento orbital	Trayectoria de portales en nivel 2

Cuadro 8: Fórmulas físicas de PhysicsEngine

F1 $x(t) = x_0 + v_{0x}t, \quad y(t) = y_0 + v_{0y}t - \frac{1}{2}gt^2$

F2 $\text{radio}(t) = A \cdot \sin(\omega t)$

F3 $v(t) = v_0 + a \cdot \Delta t, \quad v \leq v_{\text{máx}}$

$$\mathbf{F4} \quad v_f = -e \cdot v_i$$

$$\mathbf{F5} \quad x(t) = c_x + r \cos(\omega t + \varphi), \quad y(t) = c_y + r \sin(\omega t + \varphi)$$

4.3. Clase DifficultyConfig

```

1 static DifficultyConfig facil();
2 static DifficultyConfig normal();
3 static DifficultyConfig dificil();

```

Listing 6: Patrón fábrica de dificultad

Parámetro	Fácil	Normal	Difícil	Efecto
gravedadG	200	300	450	Altura de la parábola
amplitudMAS	30	50	80	Tamaño del portal
velBaseLevy	80	120	185	Velocidad de Levy
cooldownEvasion	3.5	2.0	0.7	Frecuencia de teleportación
agresividadIA	0.7	1.0	1.5	Agresividad inicial
vidaJugador	500	400	260	Vida inicial del jugador

Cuadro 9: Parámetros comparativos de dificultad

4.4. Clase GameState

Contiene: nivelActual, puntos, tiempoRestante, vidaJugador, vidaLevy, y el enum EstadoPartida con valores MENU, JUGANDO, PAUSA, VICTORIA y DERROTA.

5. Capa de presentación

5.1. Clase MainWindow

- **Game loop:** QTimer dispara onUpdate() cada 16 ms. Calcula dt con QElapsedTimer.
- **Input:** QSet<int> de teclas presionadas. procesarInput() traduce el estado a comandos.
- **Sincronización:** sincronizarSprites() lee posiciones del engine y mueve los QGraphicsPixmapItem.
- **HUD:** actualizarHUD() actualiza barras de vida y marcador leyendo GameState.
- **Menús:** gestiona menú principal, pausa y dificultad con listas de QGraphicsItem*.

6. Estructuras de datos

Estructura	Clase	Propósito
QList<Enemigo*>	GameEngine	Lista polimórfica de enemigos
QList<Portal*>	GameEngine	Lista de portales activos
deque<int8_t>	AngstromLevy	Ventana deslizante de 10 direcciones
float[9] estático	VariantePortal	Pesos compartidos entre todas las instancias
QSet<int>	MainWindow	Teclas presionadas, consulta O(1)

Cuadro 10: Estructuras de datos del proyecto

7. Conclusiones

El proyecto Invincible Kickboxer aplica de forma coherente los principios de programación orientada a objetos. La jerarquía de herencia con dos clases abstractas (**Entidad** y **Enemigo**) permite reutilizar código y aplicar polimorfismo dinámico. El agente de **AngstromLevy** demuestra que un sistema de inteligencia artificial funcional puede construirse con estructuras simples como una **deque** y una máquina de estados. La separación entre la capa de lógica y la capa de presentación garantiza que el proyecto sea mantenible y extensible.